CHSH Inequaliy Homework

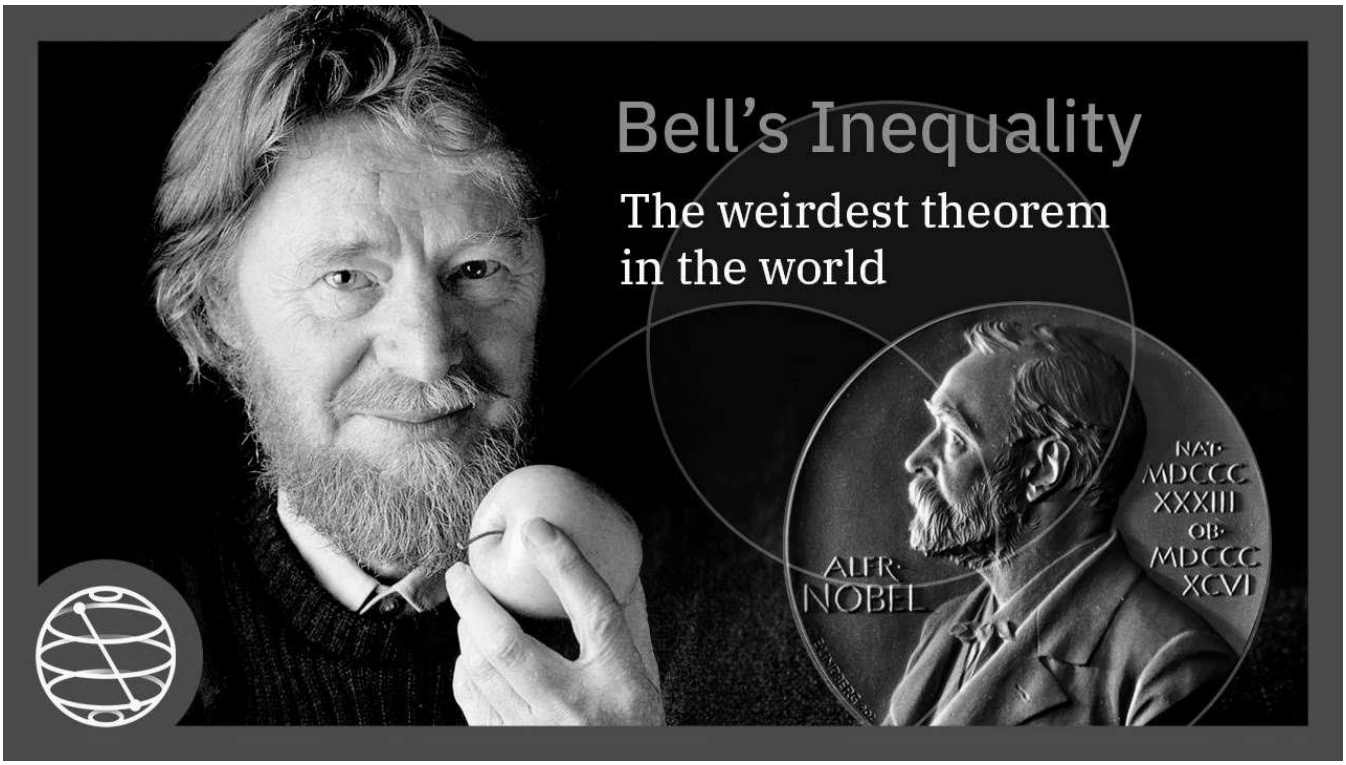
PLEASE READ ALL SECTIONS. DO NOT REMOVE ANY MARKDOWN OR CODE CELLS.

Overview

In this tutorial, you will run an experiment on a quantum computer to demonstrate the violation of the CHSH inequality.

The CHSH (or Bell) Inequality, named after the authors Clauser, Horne, Shimony, and Holt, is used to experimentally prove Bell's theorem (1969). This theorem asserts that local hidden variable theories cannot account for some consequences of entanglement in quantum mechanics. The violation of the CHSH inequality is used to show that quantum mechanics is incompatible with local hidden-variable theories. This is an important experiment for understanding the foundation of quantum mechanics.

The 2022 Nobel Prize for Physics was awarded to Alain Aspect, John Clauser and Anton Zeilinger in part for their pioneering work in quantum information science, and in particular, for their experiments with entangled photons demonstrating violation of Bell's inequalities.



You can learn more about it in this video

Objectives

- Create an open (free) account for IBM Quantum Learning
- Use the Qiskit API to build and run simple circuits
- Use quantum circuit results to justify the violation of the CHSH inequality

Grading

You are expected to write your own code. It is acceptable to find and use code from the Qiskit API and IBM Quantum Learning webpages, but **it is strictly prohibited to use code obtained from other sources**, especially other students. You are also expected to write descriptive comments explaining what your code is trying to accomplish and why.

These are the following gradable items and their weights

1. Building Circuits (40%)

    A. Entangling Qubits (12%)
        - Build Bell Circuit (10%)
        - Draw Bell Circuit (2%)
    B. Adding View Measurements (28%)
        - Building Experiments (20%)
        - Plotting View Circuits (8%)
2. Running Circuits (40%)

    A. Create Plotting Function (5%)
    B. Run Circuit on Ideal Simulator (10%)
    C. Run Circuit on Noisy Simulator (10%)
        - Get a Backend (1%)
        - Creating a Noise Model (2%)
        - Transpile Circuits to Backend (2%)
        - Run and Plot Results (5%)
    D. Run Circuit on IBM Quantum Computer (15%)
        - Parallelize Experiments (7%)
        - Transpile Circuit (2%)
        - Run Circuit (2%)
        - Plot Results (4%)
3. Calculations (20%)

    A. Calculating Expectation Values (8%)
        - Function to Calculate Expectation Value (1%)
        - Calculate Expectation Value for Ideal Simulator (2%)
        - Calculate Expectation Value for Noisy Simulator (2%)
        - Calculate Expectation Value for IBM Quantum Computer (3%)
    B. Calculating CHSH Values (12%)
        - Function to Calculate CHSH Value (2%)
        - Calculate CHSH Value for Ideal Simulator (3%)
        - Calculate CHSH Value for Noisy Simulator (3%)
        - Calculate CHSH Value for IBM Quantum Computer (4%)

Environment Verification

Before you proceed, please verify your kernel is running the **Python 3.12** environment you created with

`pip install -r requirements.txt`

If you don't see your environment in the kernel list, you need to add it using the terminal and relaunch jupyter. You can add your environment to interactive python using this command

`ipython kernel install --user --name=ENV_NAME` (replacing `ENV_NAME`)

After you've selected the correct environment, run this next cell, but do NOT edit it in any way! If no errors pop up, then your environment should be able to run this notebook with ease.

In [ ]:
```
# DO NOT EDIT THIS CELL
n = 20

import sys
print("python", sys.version)

import qiskit
print("qiskit".ljust(n), qiskit.__version__)

import qiskit_aer
print("qiskit_aer".ljust(n), qiskit_aer.__version__)

import qiskit_ibm_runtime
print("qiskit_ibm_runtime".ljust(n), qiskit_ibm_runtime.__version__)

import numpy
print("numpy".ljust(n), numpy.__version__)

import matplotlib
print("matplotlib".ljust(n), matplotlib.__version__)
```

```
python 3.12.10 (tags/v3.12.10:0cc8128, Apr  8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)]
qiskit               2.2.2
qiskit_aer           0.17.1
qiskit_ibm_runtime   0.43.0
numpy                2.4.3
matplotlib           3.10.0
```

Library Imports

We will need these libraries, classes, and utilities to write and run code for this homework. There should be no import errors if everything is installed correctly.

In [ ]:
```
# General
import numpy as np
from typing import Optional

# Plotting routines
import matplotlib.pyplot as plt

# Qiskit imports
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
from qiskit.visualization import plot_histogram
from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager

# Qiskit Aer imports
from qiskit_aer import AerSimulator
```

```
from qiskit_aer.noise import NoiseModel

# Qiskit Runtime imports
from qiskit_ibm_runtime import QiskitRuntimeService, Session, SamplerV2 as Sampler, SamplerOptio
```

Section 1: Building Circuits (40%)

This section is dedicated to building quantum circuits using Qiskit

Part 1: Creating Entangled Qubits (12%)

These experiments will require applying rotations and measurements on entangled qubits, so let's start by writing a Python function that will create and return a 2-qubit 2-cbit circuit that applies a Hadarmard gate on the first qubit and a CNOT targetting the second qubit controlled by the first qubit. An optional barrier can be added to make visualization cleaner.

Building a Bell Circuit (10%)

Fill in the Python function `BellCircuit` with Qiskit code that will return a quantum circuit created from `qreg` and `creg` that applies a Hadarmard gate on the first qubit and a CNOT targetting the second qubit controlled by the first qubit. An optional barrier can be added to make visualization cleaner. Do NOT perform any measurements at this stage!

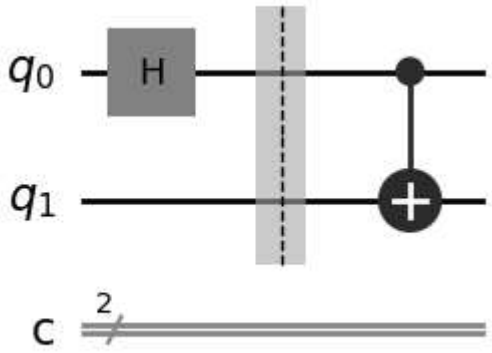
In [ ]:
```
# Python function that creates a maximally entangled bell state
def BellCircuit(qreg_name: Optional[str]='q', creg_name: Optional[str]='c') -> QuantumCircuit:
    qreg_q = QuantumRegister(2, qreg_name)
    creg_c = ClassicalRegister(2, creg_name)

    # YOUR CODE BELOW

    # Convert registers to single circuit
    circuit = QuantumCircuit(qreg_q, creg_c)

    # Apply gates to circuit
    circuit.h(qreg_q[0])
    circuit.barrier()
    circuit.cx(qreg_q[0], qreg_q[1])

    # Return the circuit
    return circuit

    # YOUR CODE ABOVE

bell = BellCircuit()
```

Drawing the Circuit (2%)

Use Qiskit API to `draw` the bell circuit using the matplotlib '`mpl`' style

In [ ]:
```
# YOUR CODE BELOW
bell.draw(output="mpl")
```

Out[ ]:



Part 2 Adding View Measurements (28%)

Now that we have a maximally entangled bell state, let us measure it in 4 different views *AB*, *Ab*, *aB*, and *ab*.

- *AB*: Rotate qubit 1 by around the Y-axis by $-\frac{\pi}{4}$ measuring both qubits in the Z basis
- *Ab*: Rotate qubit 1 by around the Y-axis by $\frac{\pi}{4}$ measuring both qubits in the Z basis
- *aB*: Rotate qubit 1 by around the Y-axis by $-\frac{\pi}{4}$ measuring qubit 0 in the X basis and qubit 1 in the Z basis
- *ab*: Rotate qubit 1 by around the Y-axis by $\frac{\pi}{4}$ measuring qubit 0 in the X basis and qubit 1 in the Z basis

Building Experiments (20%)

Fill in the Python function `MeasureWithView` with Qiskit code that will create these different view circuits leveraging the previous `BellCircuit` function, then plot them using the given code below.

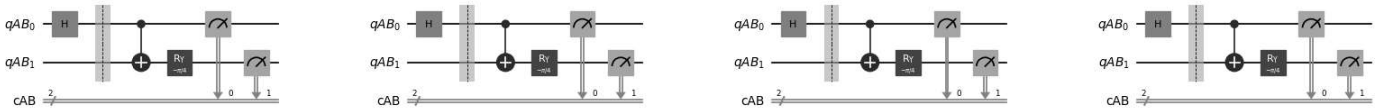
In [ ]:
```
# Write code that will create circuits for AB, Ab, aB, and ab views
views = ("AB","Ab","aB","ab")


def MeasureWithView(circuit: QuantumCircuit, view: str):
    # YOUR CODE BELOW

    # Initialize quantum and classical registers
    qreg = circuit.qregs[0]
    creg = circuit.cregs[0]

    # Create each view respective to name with respective y-axis rotations
    if view == "AB":
        circuit.ry(-(np.pi/4), qreg[1])

    elif view == "Ab":
        circuit.ry(np.pi/4, qreg[1])

    elif view == "aB":
        # Perform hadamard gates on qubit 0 to measure it in X basis
        circuit.h(qreg[0])
        circuit.ry(-(np.pi/4), qreg[1])

    elif view == "ab":
        # Perform hadamard gates on qubit 0 to measure it in X basis
        circuit.h(qreg[0])
```

```
        circuit.ry(np.pi/4, qreg[1])

    # Measure both bits into classical bits
    circuit.measure(qreg[0], creg[0])
    circuit.measure(qreg[1], creg[1])

    return circuit


circuits = [MeasureWithView(BellCircuit('q'+view,'c'+view),view) for view in views]
```

Plotting the Different View Circuits (8%)

Fill in the `PlotCircuits` Python function with code that will create a figure containing a drawing of each view circuit by calling its `draw` method. Check that your circuits make sense and appear as described in the instructions.

In [ ]:
```
# Plot the circuits into 1 figure
def PlotCircuits(*circuits: QuantumCircuit):
    # YOUR CODE BELOW

    # Initialize figure with one axis per circuit
    fig, axs = plt.subplots(1, 4, figsize=(20, 5))

    # Iterate over circuits and draw each one on corresponding axis
    count = 0
    for ax in axs :
        circuits[0].draw(output="mpl", ax = ax)
        count += 1

    # YOUR CODE ABOVE
    plt.show()

PlotCircuits(*circuits)
```



Section 2: Running Circuits (40%)

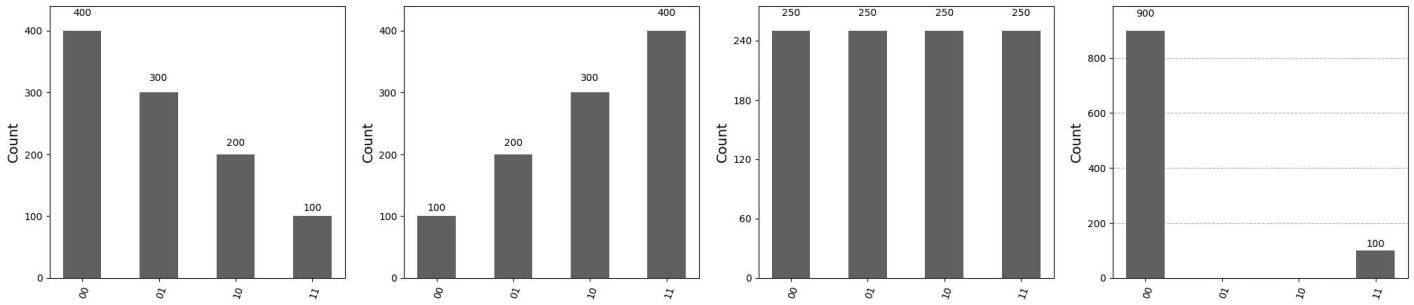
This next section is dedicated to running your quantum circuits on both ideal and noisy simulators, and finally plotting the results.

Part 1: Creating a Plotting Function (5%)

Fill the `PlotCircuitResults` Python function with code that will utilize `plot_histogram` to visualize the counts of each observable from a circuit job. There is an example dictionary you should test with.

In [ ]:
```
# Plot the circuits into 1 figure
def PlotCircuitResults(circuit_results: dict):

    # YOUR CODE BELOW
```

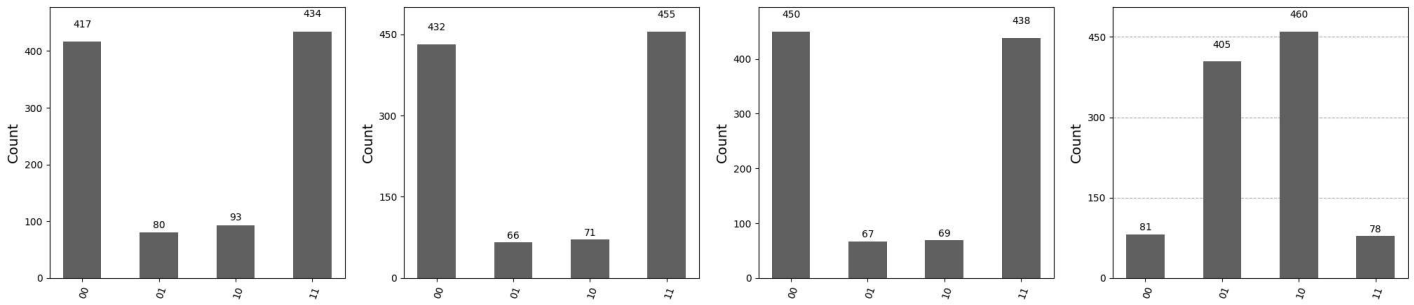
```
    # Initialize figure with one axis per circuit_results plot (as above)
    fig, axs = plt.subplots(1, 4, figsize=(25, 5))

    # Iterate over circuit_results and draw each set of results on corresponding axis
    count = 0
    for ax in axs :
        plot_histogram(circuit_results[views[count]], ax = ax)
        count += 1

    # YOUR CODE ABOVE

    plt.show()

PlotCircuitResults({
    "AB":{"00": 400, "01": 300, "10": 200, "11": 100},
    "Ab":{"00": 100, "01": 200, "10": 300, "11": 400},
    "aB":{"00": 250, "01": 250, "10": 250, "11": 250},
    "ab":{"00": 900, "01": 000, "10": 000, "11": 100},
})
```



Part 2: Running Circuits on an **Ideal Simulator** (10%)

Use Qiskit's AerSimulator to simulate 1000 shots for each view circuit, then use the `PlotCircuitResults` function to plot the counts from each view circuit job result.

In [ ]:
```
# YOUR CODE BELOW

# Initialize AerSimulator
simulator = AerSimulator()
# Create dict with circuit for each view
all_circuits = {view: MeasureWithView(BellCircuit('q'+view,'c'+view),view) for view in views}
# Create dict for all simulator results
results = {view: simulator.run(all_circuits[view]).result() for view in views}
# Create dict for the counts of the simulator's results
all_counts = {view: results[view].get_counts() for view in views}
PlotCircuitResults(all_counts)
```



Part 3: Running Circuits on an **IBM Noisy Simulator** (10%)

Use Qiskit's AerSimulator and NoiseModel to simulate 1000-shot runs of each view circuit as if they were ran on an 127-qubit IBM quantum computer, then use the `PlotCircuitResults` function to plot the quasi-dists from each view circuit job result.

Loading Your QiskitRuntimeService Account

To request a backend from IBM, you need an IBM Quantum account synced to this notebook. For a service (like IBM) to track users jobs and utilization of their hardware resources, Qiskit uses an API token, CRN, and instance which are tied to your IBM Quantum account. It is critical to keep it secret because any device could use it and use QPU time under your account which would drain your credits / computing funds. See the QiskitRuntimeService for more info.

In [ ]:
```
# Copy your token from the IBM Quantum Platform, run this cell, then delete the token.
QiskitRuntimeService.save_account(
    token="",
    instance="crn:v1:bluemix:public:quantum-computing:us-east:a/a4a05d90379f42b9ae5312af6b0dcc2c
    set_as_default=True,
    # Use `overwrite=True` if you're updating your token.
    overwrite=True,
)
```

Once you run the cell above, you only need to run the cell below to get the service because your API token is now safely stored on your device, but keep in mind if you run this notebook on another device, the saved account may be different.

In [ ]:
```
# Load saved credentials
service = QiskitRuntimeService()
```

```
management.get:WARNING:2026-04-02 12:49:07,270: Loading default saved account
```

Getting a Backend (1%)

Use the QiskitRuntimeService to request a real 127-qubit quantum computer backend from the `ibm_quantum` channel.
PLEASE DO NOT EXPLICITLY WRITE YOUR API TOKEN IN THIS NOTEBOOK, BUT RATHER SAVE YOUR ACCOUNT

In [ ]:
```
# YOUR CODE BELOW
backend = service.backend('ibm_fez')
```

Run the cell below to open a web browser to view the backend you requested

In [ ]:
```
import webbrowser
webbrowser.open(f"https://quantum.cloud.ibm.com/computers?system={backend.name}")
```

Out[ ]:
```
True
```

Creating a Noise Model (2%)

Create a `NoiseModel` using the backend you just requested, then create an `AerSimulator` using that noise model.

In [ ]:
```
# YOUR CODE BELOW
noise_model = NoiseModel.from_backend(backend)
sim = AerSimulator(noise_model=noise_model)
```

Transpile Circuit for Noisy Backend (2%)

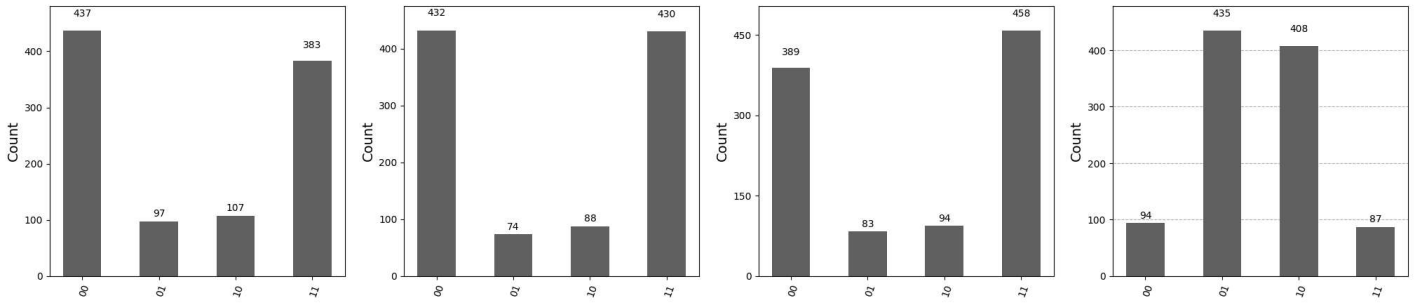
Use the `general_preset_pass_manager` to create a transpiler for your backend, then transpile your circuits.

In [ ]:
```
# YOUR CODE BELOW
# Create pass manager
pass_manager = generate_preset_pass_manager(optimization_level=3, backend=sim)
# Transpile all four circuits
all_transpiled = {view: pass_manager.run(all_circuits[view]) for view in views}
```

Run Transpiled Circuits and Plot Results (5%)

Run the transpiled circuits using the noisy simulator, then call `PlotCircuitResults` plot the counts.

In [ ]:
```
# YOUR CODE BELOW
# Create dict with all noisy run results
transpiled_results = {view: sim.run(all_transpiled[view]).result() for view in views}
# Create dict for the counts of the noisy results
transpiled_counts = {view: transpiled_results[view].get_counts() for view in views}
PlotCircuitResults(transpiled_counts)
```



Part 4: Running Circuits on an IBM Quantum Computer (15%)

Running a circuit on IBM's quantum hardware is free but limited by Quantum Processing Unit (QPU) time per month. As such, caution should be taken before sending jobs to a real device. If the job is not set up correctly, it can easily result in a waste of QPU time.

When a job is sent for execution, it enters a job queue because a QPU can only service one job at a time and thousands of users are submitting jobs daily. QPU time is only accumulated when the job starts running after it has finished queueing.

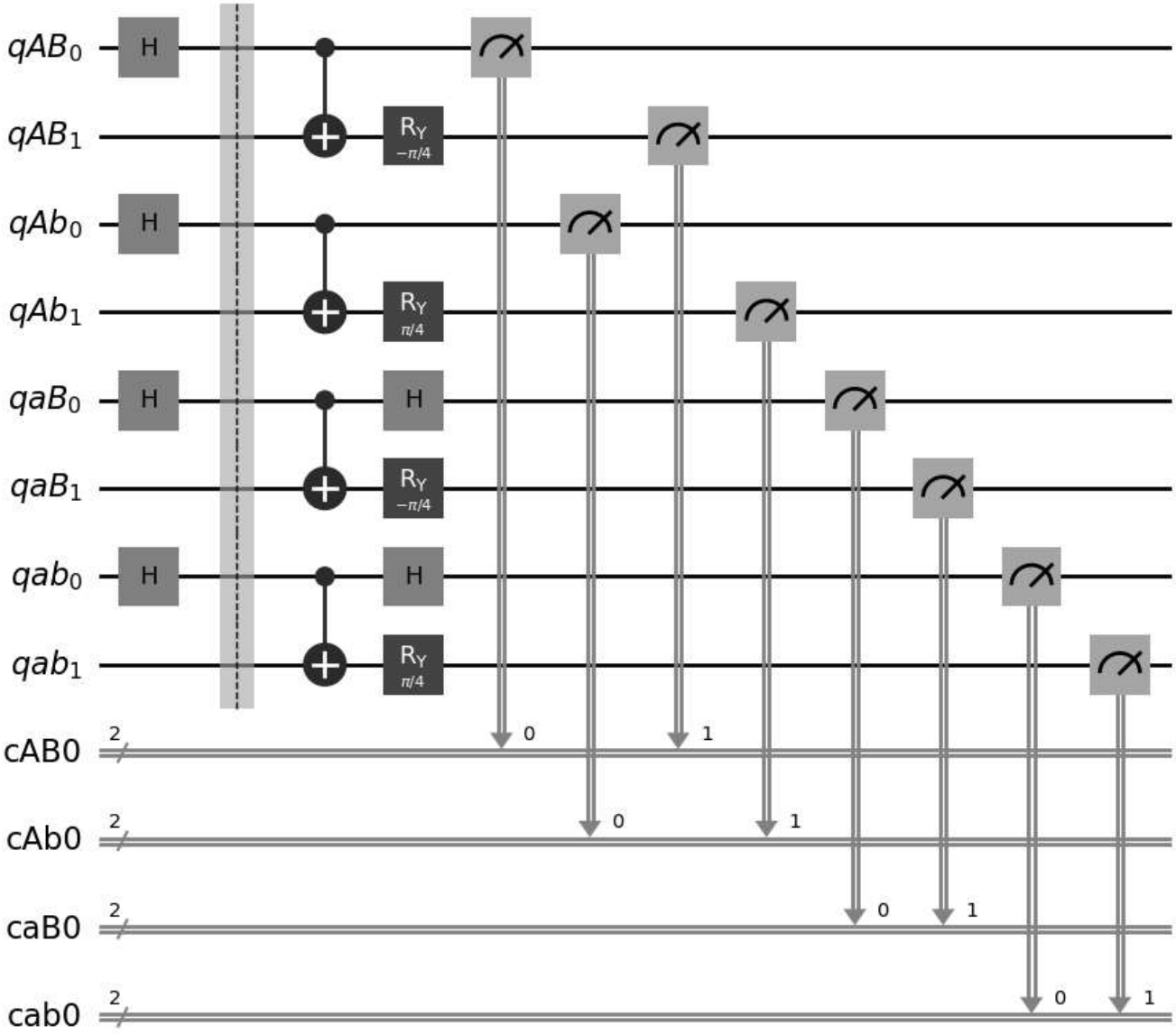
Optimizing Circuits for Real Backend

We have 4 experiments that utilize 2 qubits each, but our real backend uses quantum hardware that supports over 100 qubits. To be more efficient with that computing resource, we should parallelize some shots by using more qubits to save QPU time.

To achieve this, we will create a new 8-qubit circuit that runs all 4 experiments simulaneously on disjoint pairs of qubits. This will cut our QPU usage down by 75% since we are effectively parallelizing the experiments. Just run the code below to see how this works.

In [ ]:
```
# This code creates a wider QuantumCircuit object that concatenates each view experiment circuit
circ_combined = QuantumCircuit()

for j, view in enumerate(views):
    # Note that having a custom classical register name for each experiment
    # will be really helpful for post-processing counts from a sampler
    circuit = MeasureWithView(
        BellCircuit('q'+view,'c'+view+'0'),
        view
    )
    circ_combined.add_register(circuit.qregs[0])
    circ_combined.add_register(circuit.cregs[0])
    circ_combined.compose(
        circuit,
        qubits=range(2*j, 2*(j+1)),
        clbits=range(2*j, 2*(j+1)),
        inplace=True
    )

circ_combined.draw('mpl')
```

Out[ ]:



Parallelizing Copies of Experiments (7%)

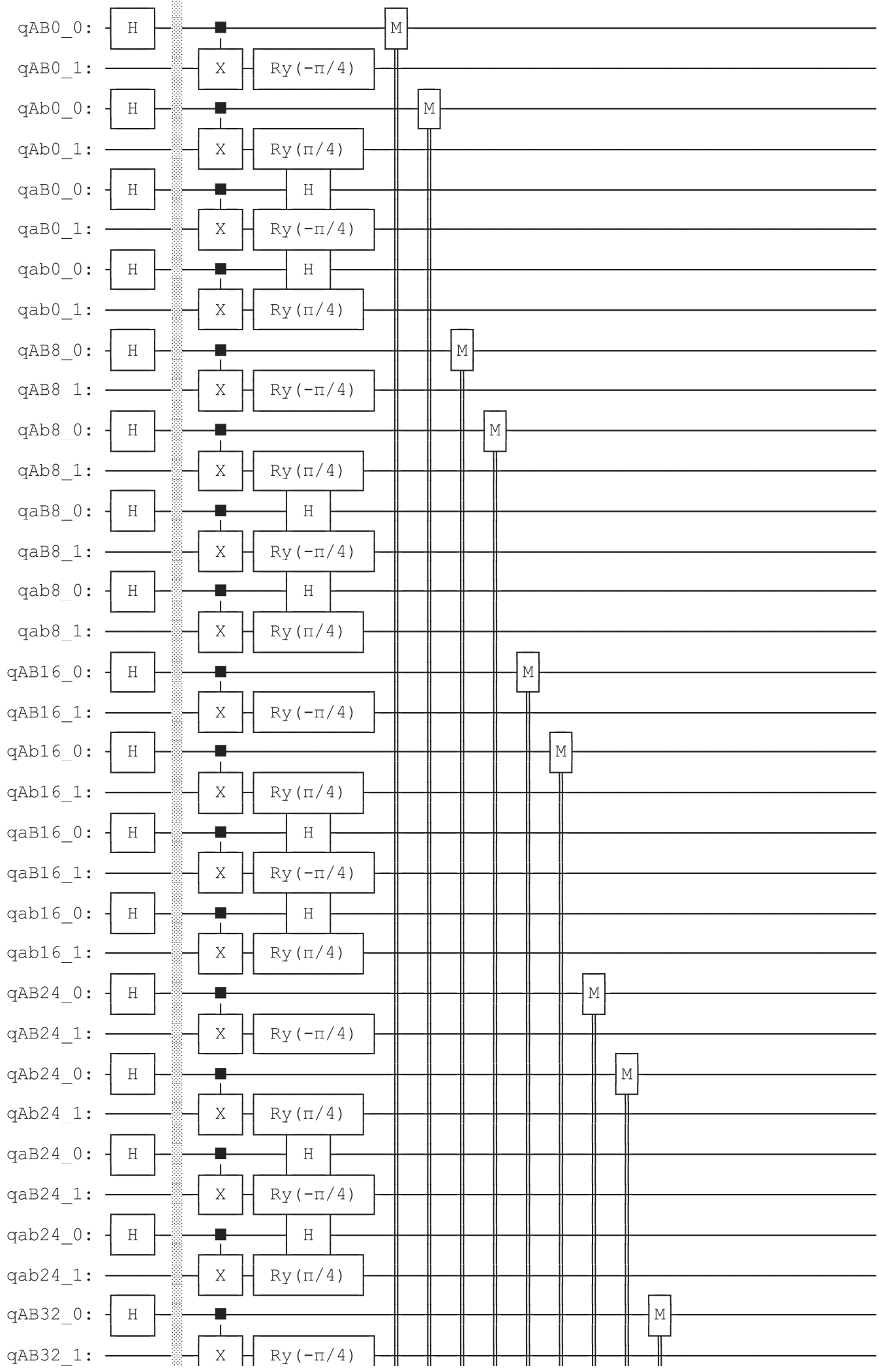
Now we have a circuit that will execute all 4 experiments side by side, but this uses only 8 qubits. We can still utilize more of the hardware resources by creating copies of this circuit across the entire qubit topology.

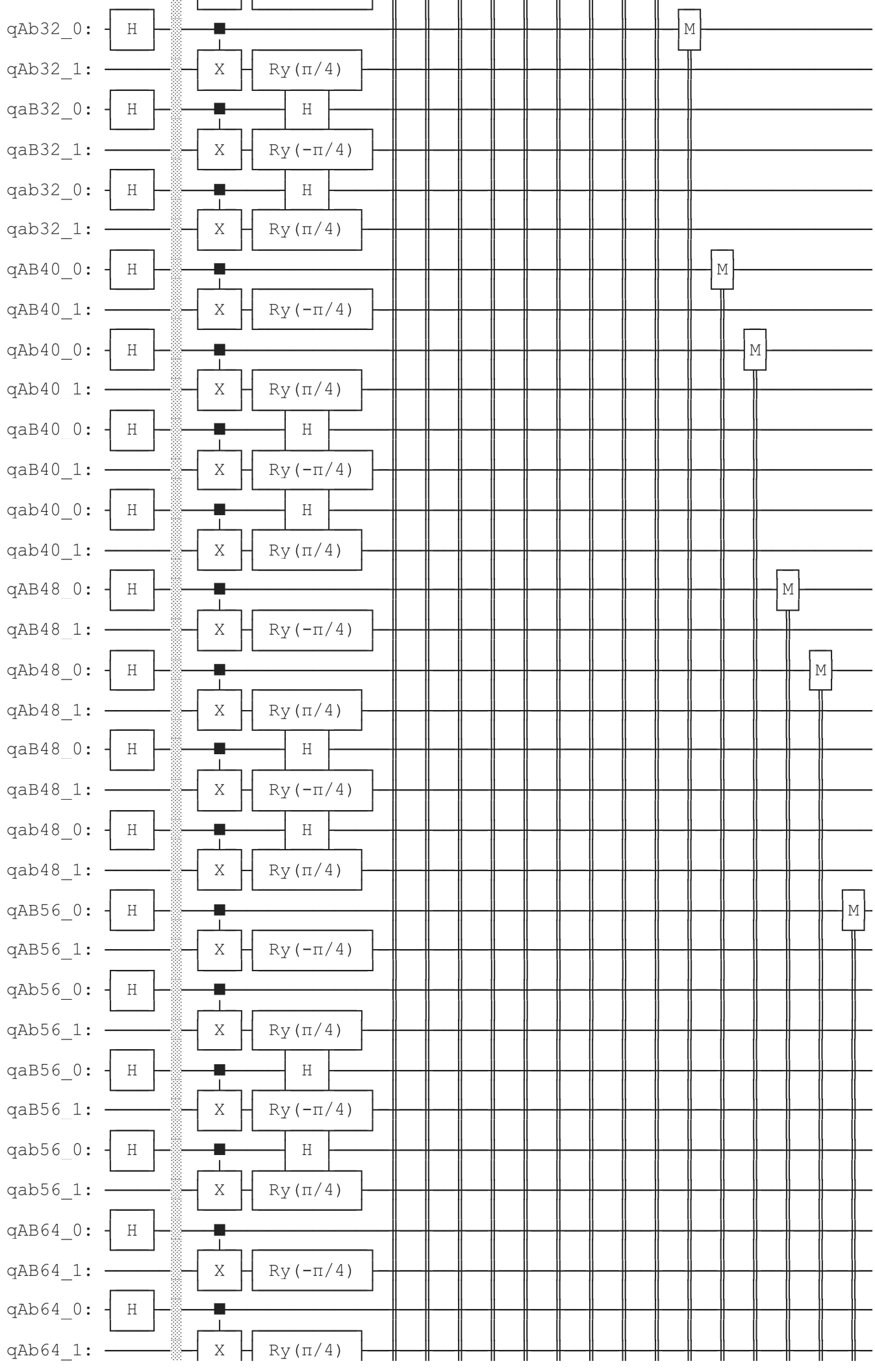
Fill in the rest of the code to create one wide, shallow circuit containing several copies of each measurement view experiment. Be sure to provide distict names for the classical and quantum registers in each experiment circuit. This will make it easier to process later. You can refer to the previous code cell for guidance.

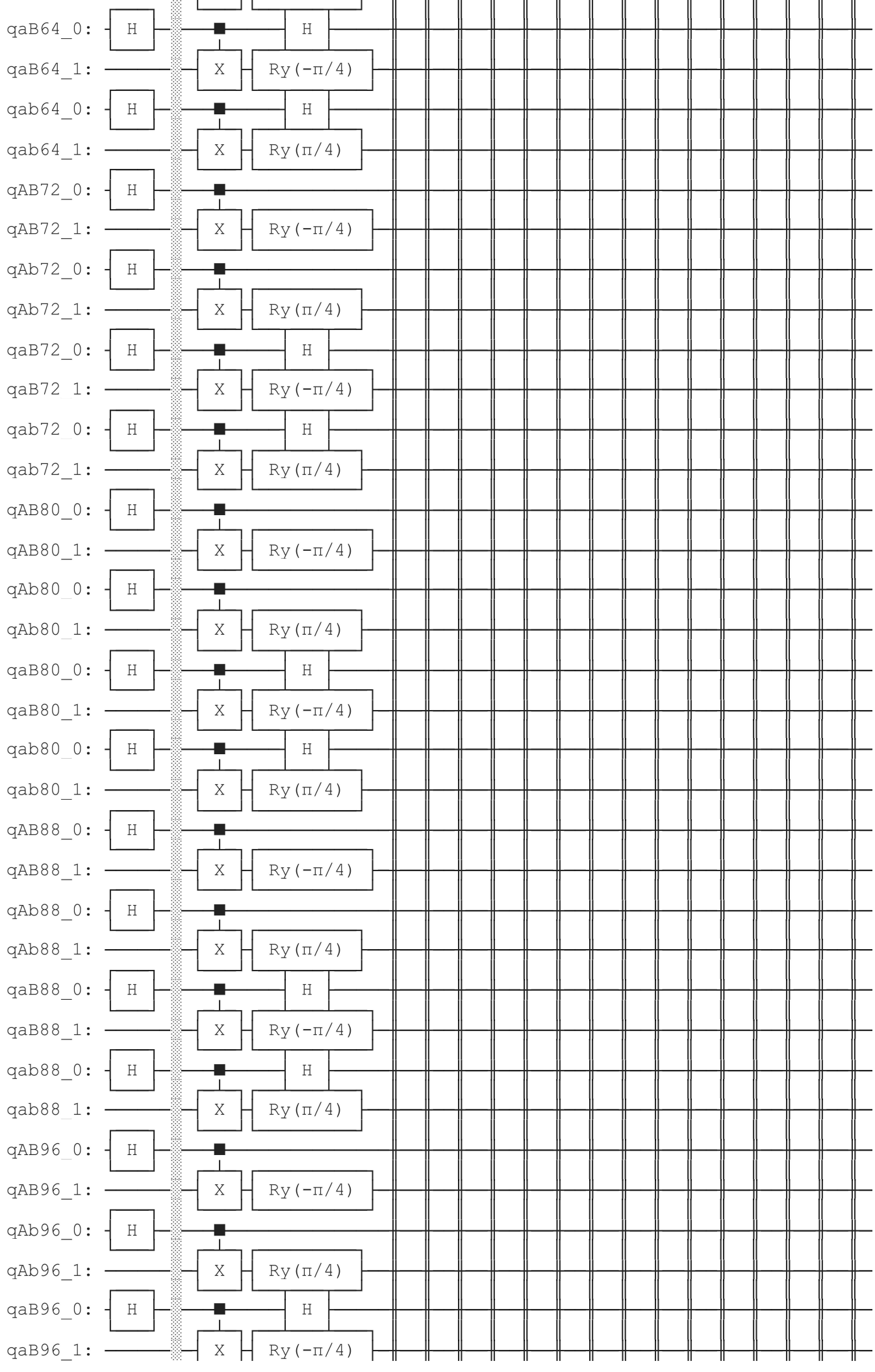
In [ ]:
```
# In a similar manner, create an even wider circuit using copies of the combined experiments circ
# Utilize roughly 80% of the machine to accomodate broken connections between qubits and toplogy
num_qubits = 8 * (int(0.8*backend.num_qubits) // 8)
circ_real = QuantumCircuit()
for i in range(0,num_qubits,8):
    for j, view in enumerate(views):
        # YOUR CODE BELOW
        # create separate circuit copies with all views and add their registers to larger circuit
        # names of registers are based on view name and circuit number
        individual_circuit = MeasureWithView(BellCircuit(f"q"+view+str(i), f"c"+view+str(i//8)),
        circ_real.add_register(individual_circuit.qregs[0])
        circ_real.add_register(individual_circuit.cregs[0])
```

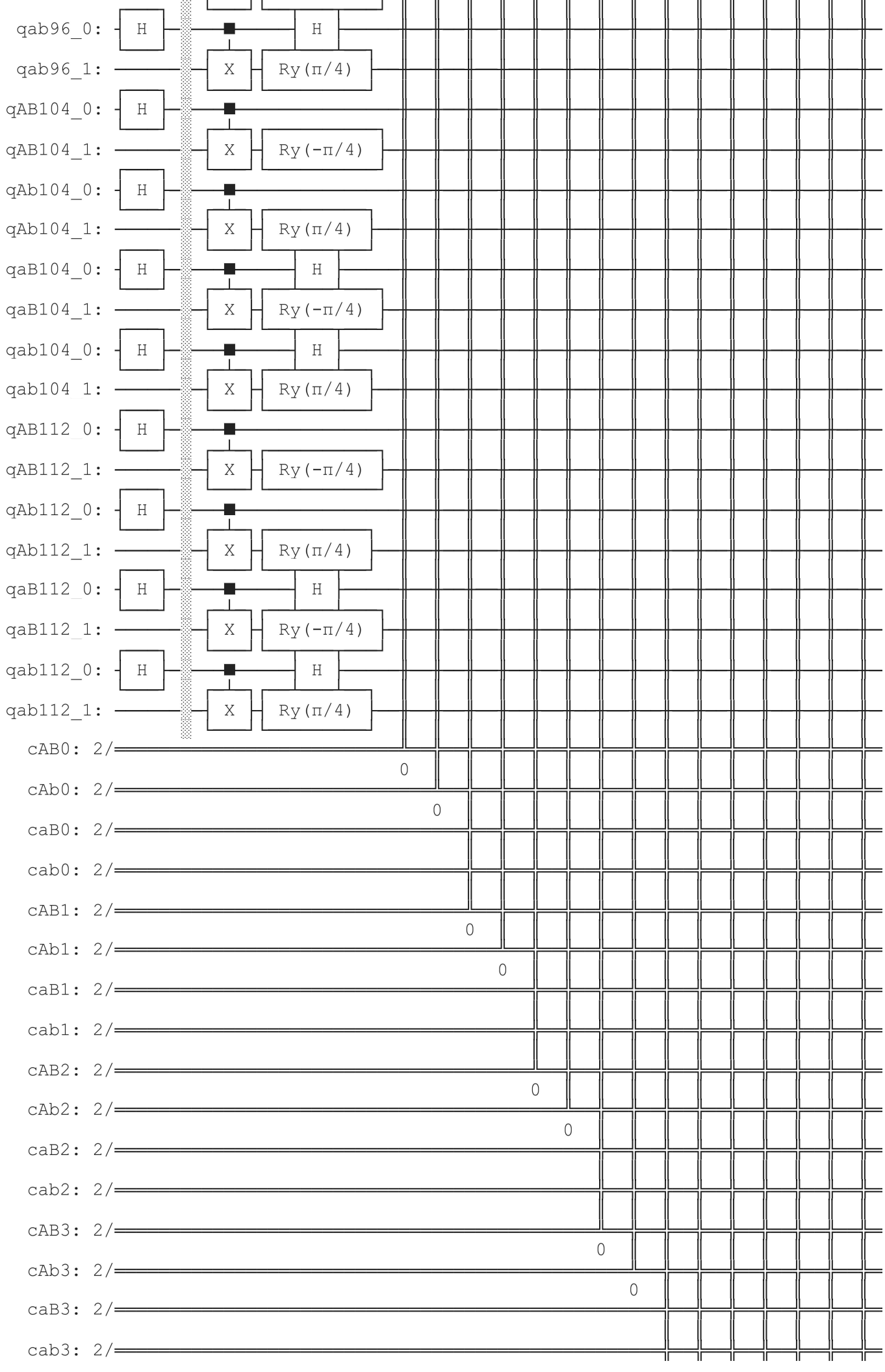
```
        # Compose circuit with offsets for each circuit
        circ_real.compose(
            individual_circuit,
            qubits=range(i+2*j, i+2*(j+1)),
            clbits=range(i+2*j, i+2*(j+1)),
            inplace=True
        )

circ_real.draw(fold=-1)
```

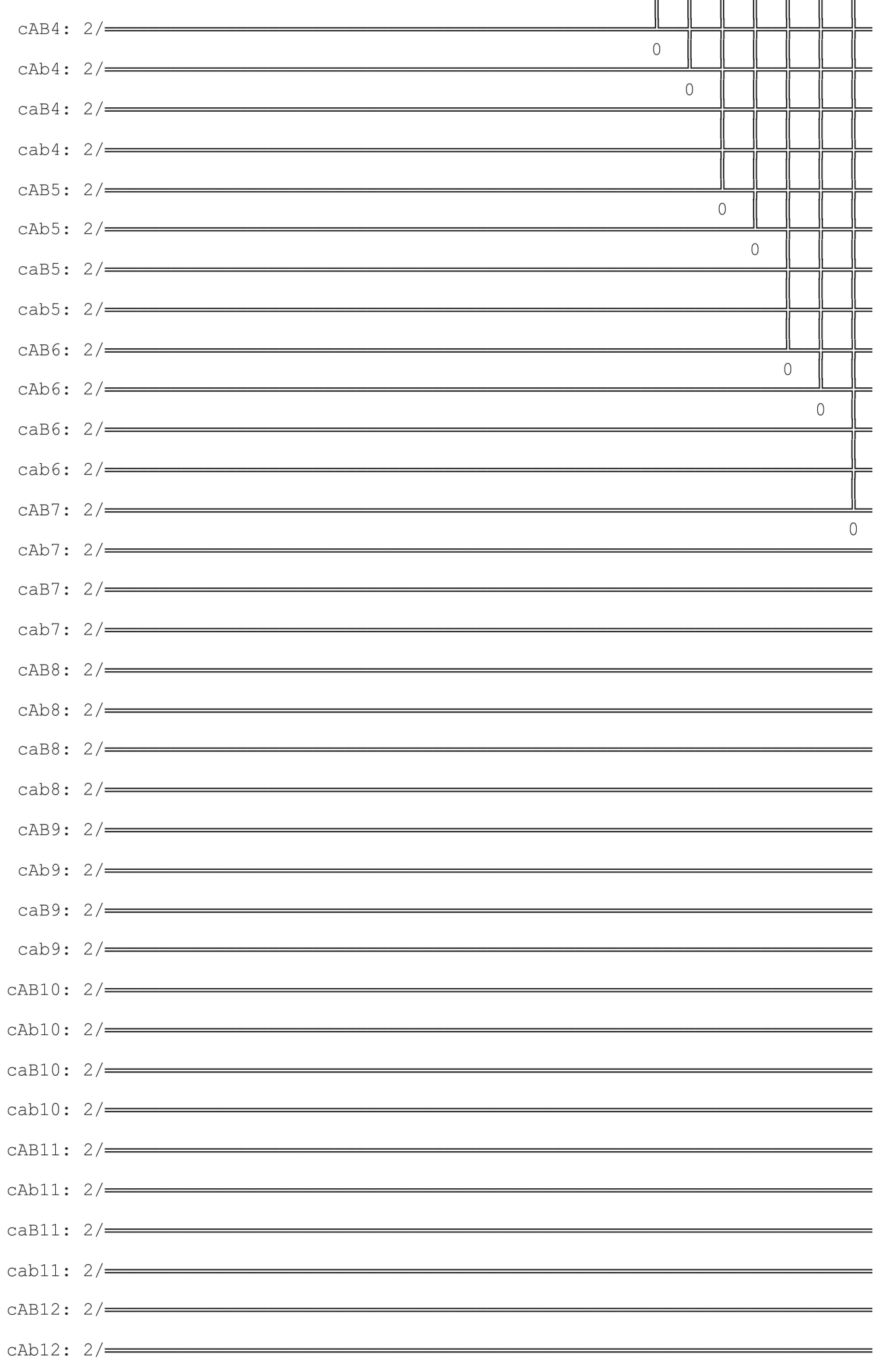
Out[ ]:


qAB0_0:
qAB0_1:
qAb0_0:
qAb0_1:
qaB0_0:
qaB0_1:
qab0_0:
qab0_1:
qAB8_0:
qAB8 1:
qAb8_0:
qAb8_1:
qaB8_0:
qaB8_1:
qab8_0:
qab8_1:
qAB16_0:
qAB16_1:
qAb16_0:
qAb16_1:
qaB16_0:
qaB16_1:
qab16_0:
qab16_1:
qAB24_0:
qAB24_1:
qAb24_0:
qAb24_1:
qaB24_0:
qaB24_1:
qab24_0:
qab24_1:
qAB32_0:
qAB32_1:
H
X
Ry(-π/4)
Ry(π/4)
M


qAb32_0:
qAb32_1:
qaB32_0:
qaB32_1:
qab32_0:
qab32_1:
qAB40_0:
qAB40_1:
qAb40_0:
qAb40 1:
qaB40 0:
qaB40_1:
qab40_0:
qab40_1:
qAB48 0:
qAB48_1:
qAb48_0:
qAb48_1:
qaB48_0:
qaB48_1:
qab48_0:
qab48_1:
qAB56_0:
qAB56_1:
qAb56_0:
qAb56_1:
qaB56_0:
qaB56_1:
qab56_0:
qab56_1:
qAB64_0:
qAB64_1:
qAb64_0:
qAb64_1:
H
X
Ry(π/4)
Ry(-π/4)
M


qaB64_0:
H
H
qaB64_1:
X
Ry(-π/4)
qab64_0:
H
H
qab64_1:
X
Ry(π/4)
qAB72_0:
H
qAB72_1:
X
Ry(-π/4)
qAb72_0:
H
qAb72_1:
X
Ry(π/4)
qaB72_0:
H
H
qaB72 1:
X
Ry(-π/4)
qab72_0:
H
H
qab72_1:
X
Ry(π/4)
qAB80_0:
H
qAB80_1:
X
Ry(-π/4)
qAb80_0:
H
qAb80_1:
X
Ry(π/4)
qaB80_0:
H
H
qaB80_1:
X
Ry(-π/4)
qab80_0:
H
H
qab80_1:
X
Ry(π/4)
qAB88_0:
H
qAB88_1:
X
Ry(-π/4)
qAb88_0:
H
qAb88_1:
X
Ry(π/4)
qaB88_0:
H
H
qaB88_1:
X
Ry(-π/4)
qab88_0:
H
H
qab88_1:
X
Ry(π/4)
qAB96_0:
H
qAB96_1:
X
Ry(-π/4)
qAb96_0:
H
qAb96_1:
X
Ry(π/4)
qaB96_0:
H
H
qaB96_1:
X
Ry(-π/4)


qab96_0:
qab96_1:
qAB104_0:
qAB104_1:
qAb104_0:
qAb104_1:
qaB104_0:
qaB104_1:
qab104_0:
qab104 1:
qAB112 0:
qAB112_1:
qAb112_0:
qAb112_1:
qaB112 0:
qaB112_1:
qab112_0:
qab112_1:
H
X
Ry(π/4)
Ry(-π/4)
cAB0: 2/
cAb0: 2/
caB0: 2/
cab0: 2/
cAB1: 2/
cAb1: 2/
caB1: 2/
cab1: 2/
cAB2: 2/
cAb2: 2/
caB2: 2/
cab2: 2/
cAB3: 2/
cAb3: 2/
caB3: 2/
cab3: 2/
0


cAB4: 2/
0
cAb4: 2/
0
caB4: 2/
cab4: 2/
cAB5: 2/
0
cAb5: 2/
0
caB5: 2/
cab5: 2/
cAB6: 2/
0
cAb6: 2/
0
caB6: 2/
cab6: 2/
cAB7: 2/
0
cAb7: 2/
caB7: 2/
cab7: 2/
cAB8: 2/
cAb8: 2/
caB8: 2/
cab8: 2/
cAB9: 2/
cAb9: 2/
caB9: 2/
cab9: 2/
cAB10: 2/
cAb10: 2/
caB10: 2/
cab10: 2/
cAB11: 2/
cAb11: 2/
caB11: 2/
cab11: 2/
cAB12: 2/
cAb12: 2/

```
caB12: 2/═══════════════════════════════════════════════════════════════
cab12: 2/═══════════════════════════════════════════════════════════════
cAB13: 2/═══════════════════════════════════════════════════════════════
cAb13: 2/═══════════════════════════════════════════════════════════════
caB13: 2/═══════════════════════════════════════════════════════════════
cab13: 2/═══════════════════════════════════════════════════════════════
cAB14: 2/═══════════════════════════════════════════════════════════════
cAb14: 2/═══════════════════════════════════════════════════════════════
caB14: 2/═══════════════════════════════════════════════════════════════
cab14: 2/═══════════════════════════════════════════════════════════════
```

Transpile the Circuit for Real Backend (2%)

Use the `general_preset_pass_manager` using level 2 optimization to create a transpiler for your backend, then transpile your circuits. Print the depth of your circuit. The depth should be fairly shallow after optimization (less than 30 layered operations).

In [ ]:
```
# YOUR CODE BELOW
# Create pass manager
pass_manager = generate_preset_pass_manager(optimization_level=2, backend=backend)
# Transpile circuit
transpiled_circuit = pass_manager.run(circ_real)
print(transpiled_circuit.depth())
```

```
11
```

Run the Transpiled Circuit (2%)

Run the transpiled circuit on real quantum hardware using the `Sampler`. Note that when you run the job, you will have to wait for the result, so it is suggested you do not call `result()` right away but rather call `job_id()`

In [ ]:
```
# YOUR CODE BELOW
sampler = Sampler(backend)
job = sampler.run([transpiled_circuit])
print(job.job_id())
```

```
d77acuk6ji0c738cjr00
```

Retrieving jobs after closing notebook

If you need to close your notebook, your job will continue queueing or running on the cloud, so when you want the results, you can use this code snippet to get your job instance back.

In [ ]:
```
# Replace job.job_id() with "<Your Job Id>"
job = service.job("d77acuk6ji0c738cjr00")
```

Checking the Job Status

You can run `job.status()` several times to monitor your job. Only when it returns `DONE` will the `job.result()` be ready.

In [ ]:
```
# Run this cell as many times as you like. Once the job is DONE, then run grab the result
job.status()
```

Out[ ]:
```
'DONE'
```

You can even see your job on IBM Quantum's website. Try running the cell below.

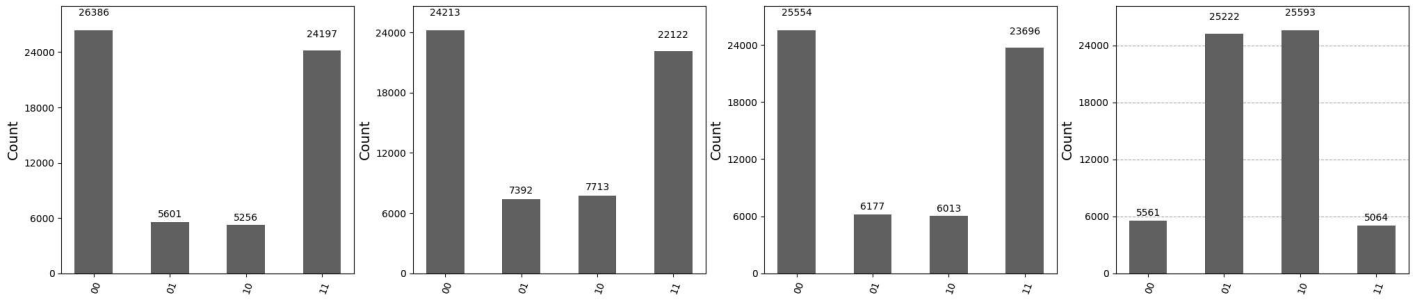
In [ ]:
```
webbrowser.open(f"https://quantum.ibm.com/jobs/{job.job_id()}")
```

Out[ ]:
```
True
```

Plotting Job Results (4%)

Aggregate the counts from each experiment to their respective view measurement, then use `PlotCircuitResults` to plot the counts.

In [ ]:
```
result = job.result()[0]

real_counts = {}
for view in views:
    real_counts[view] = {f"{q1}{q0}": 0 for q0 in range(2) for q1 in range(2)}
    for i in range(0,num_qubits//8):
        counts_dict = getattr(result.data,"c"+view+str(i)).get_counts()
        # YOUR CODE BELOW
        # Iterates over all elements of real counts and adds counts of corresponding views
        for bits, count in counts_dict.items():
            real_counts[view][bits] += count
        # YOUR CODE ABOVE

PlotCircuitResults(real_counts)
```



Section 3: Calculations (20%)

Part 1: Calculating Expectaction Values (8%)

Computing the expectation value of the circuit is given by this formula

$$\langle\psi\rangle = P(00) + P(11) - P(01) - P(10)$$

which is the **difference between** the probability of observing both qubits being the **same** and the probability the observations are **different**.

Writing a Function to Compute Expectaction Values (1%)

Fill in the `ExpectatonValue` Python function with code that will use the counts obtained from a circuit job to compute the expectation value of that circuit view.

In [ ]:
```
def ExpectationValue(counts: dict) -> float:
    # YOUR CODE BELOW
    # Summing counts and dividing by total
    total = sum(counts.values())
    return (counts["00"] + counts["11"] - counts["01"] - counts["10"])/total
```

Calculating $\langle AB \rangle$, $\langle Ab \rangle$, $\langle aB \rangle$, and $\langle ab \rangle$ from Jobs ran on an **Ideal Simulator** (2%)

Use the `ExpectationValue` function and the observable counts obtained from running your circuits on an **ideal simulator** to get an expectation value per circuit, then `print` these.

In [ ]:
```
# YOUR CODE BELOW
for view in views:
    print(view + ": " + str(ExpectationValue(all_counts[view])))
```

```
AB: 0.662109375
Ab: 0.732421875
aB: 0.734375
ab: -0.689453125
```

Calculating $\langle AB \rangle$, $\langle Ab \rangle$, $\langle aB \rangle$, and $\langle ab \rangle$ from Jobs ran on a **Noisy Simulator** (2%)

Use the `ExpectationValue` function and the observable counts obtained from running your circuits on a **noisy simulator** to get an expectation value per circuit, then `print` these.

In [ ]:
```
# YOUR CODE BELOW
for view in views:
    print(view + ": " + str(ExpectationValue(transpiled_counts[view])))
```

```
AB: 0.6015625
Ab: 0.68359375
aB: 0.654296875
ab: -0.646484375
```

Calculating $\langle AB \rangle$, $\langle Ab \rangle$, $\langle aB \rangle$, and $\langle ab \rangle$ from Jobs ran on an **IBM Quantum Computer** (3%)

Use the `ExpectationValue` function and the observable counts obtained from running your circuits on an **IBM quantum computer** to get an expectation value per circuit, then `print` these.

In [ ]:
```
# YOUR CODE BELOW
for view in views:
```

```
    print(view + ": " + str(ExpectationValue(real_counts[view])))
```

```
AB: 0.64658203125
Ab: 0.50830078125
aB: 0.6031901041666666
ab: -0.6541341145833334
```

Part 2: Calculating $\langle CHSH \rangle$ (12%)

This is the final step of the programming project. Congrats on making it this far!

Function to Calculate $\langle CHSH \rangle$ (2%)

Fill in the `CHSHValue` Python function with code that will use the expectation values from the different view circuits according this formula

$$\langle CHSH \rangle = \langle AB \rangle + \langle Ab \rangle + \langle aB \rangle - \langle ab \rangle$$

In [ ]:
```
def CHSHValue(ev: dict) -> float:
    # YOUR CODE BELOW
    return ExpectationValue(ev["AB"]) + ExpectationValue(ev["Ab"]) + ExpectationValue(ev["aB"])
```

Calculating $\langle CHSH \rangle$ from Jobs ran on a **Ideal Simulator** (3%)

Use the `CHSHValue` function and the expectation values obtained from running your circuits on a **ideal simulator** to get a CHSH value for this experiment, then `print` these and whether or not the CHSH Inequality is violated.

In [ ]:
```
# YOUR CODE BELOW
print(str(CHSHValue(all_counts)))
print("Violated: " + str(CHSHValue(all_counts) > 2))
```

```
2.818359375
Violated: True
```

Calculating $\langle CHSH \rangle$ from Jobs ran on a **Noisy Simulator** (3%)

Use the `CHSHValue` function and the expectation values obtained from running your circuits on a **noisy simulator** to get a CHSH value for this experiment, then `print` these and whether or not the CHSH Inequality is violated.

In [ ]:
```
# YOUR CODE BELOW
print(str(CHSHValue(transpiled_counts)))
print("Violated: " + str(CHSHValue(transpiled_counts) > 2))
```

```
2.5859375
Violated: True
```

Calculating $\langle CHSH \rangle$ from Jobs ran on an **IBM Quantum Computer** (4%)

Use the `CHSHValue` function and the expectation values obtained from running your circuits on an **IBM quantum computer** to get a CHSH value for this experiment, then `print` these and whether or not the CHSH Inequality is violated.

In [ ]:
```
# YOUR CODE BELOW
print(str(CHSHValue(real_counts)))
print("Violated: " + str(CHSHValue(real_counts) > 2))
```

```
2.41220703125
Violated: True
```

Submitting Your Work

If you have completed this programming homework exercise, well done!

To submit your work, please follow these steps.

1. Convert this notebook to HTML by opening a terminal, and running

   `jupyter nbconvert --to html FILEPATH_TO_NOTEBOOK` (replacing `FILEPATH_TO_NOTEBOOK`)

2. Open your HTML file in a browser
3. Right-click in the browser and click print
4. Change print settings to save as a pdf, set the margin to none, and downscale (if necessary, but don't make it too small)
5. Upload your PDF file to the Pilot dropbox